Ray

A Distributed Framework for Emerging AI Applications

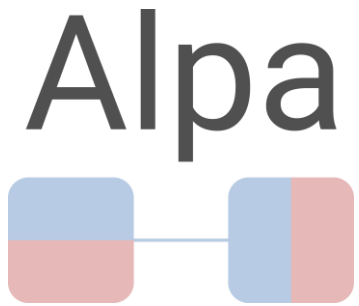
2025.07.24 이유찬

Paper Info

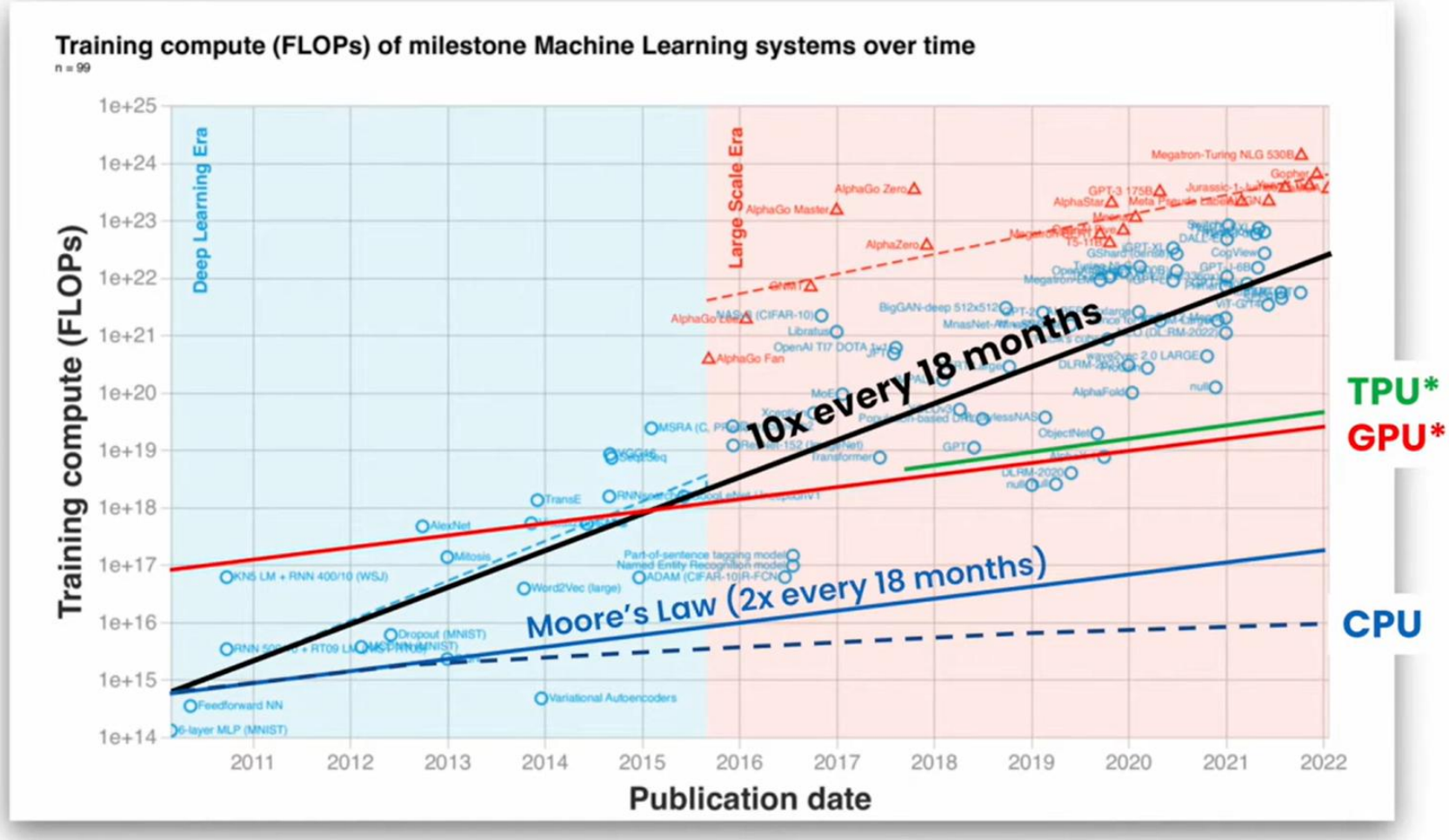
- OSDI 18

- Philipp Moritz, Robert Nishihara, Stephanie Wang, Alexey Tumanov, Richard Liaw, Eric Liang, Melih Elibol, Zongheng Yang, William Paul, Michael I. Jordan, and Ion Stoica

- UC Berkeley



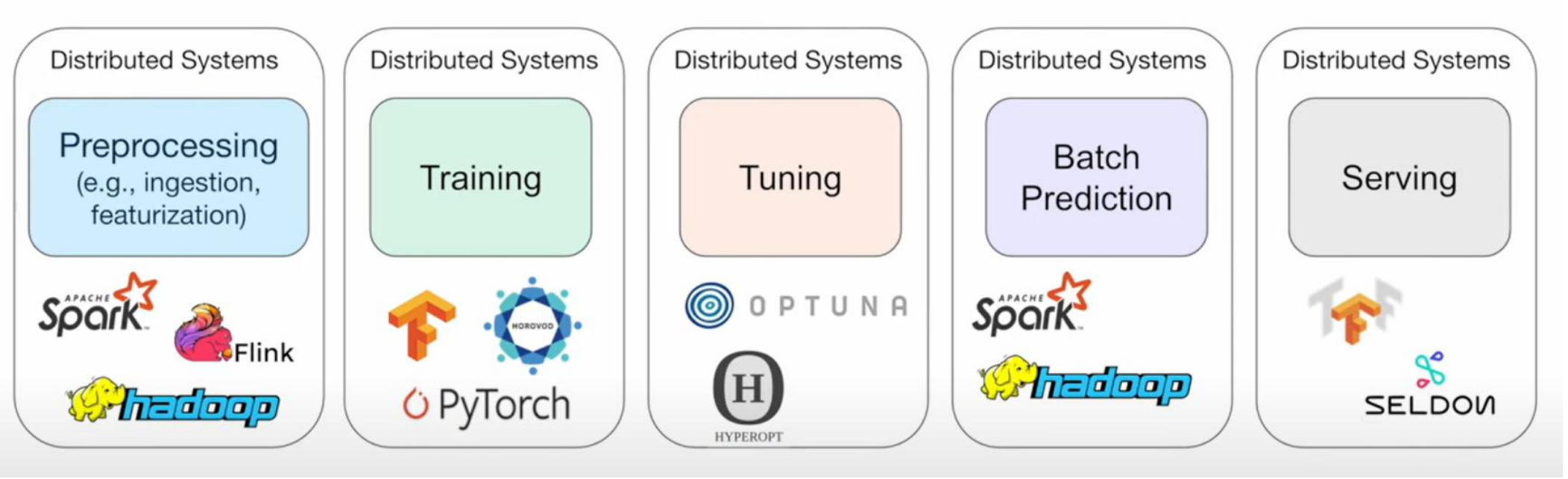
Motivation



"Compute trends across three eras of machine learning", J. Sevilla, https://ar5iv.labs.arxiv.org/html/2202.05924

Motivation

- 초기에는 RL(Reinforcement Learning)을 위해 개발된 프레임워크
  - 학습, 추론, 시뮬레이션 기능 요구

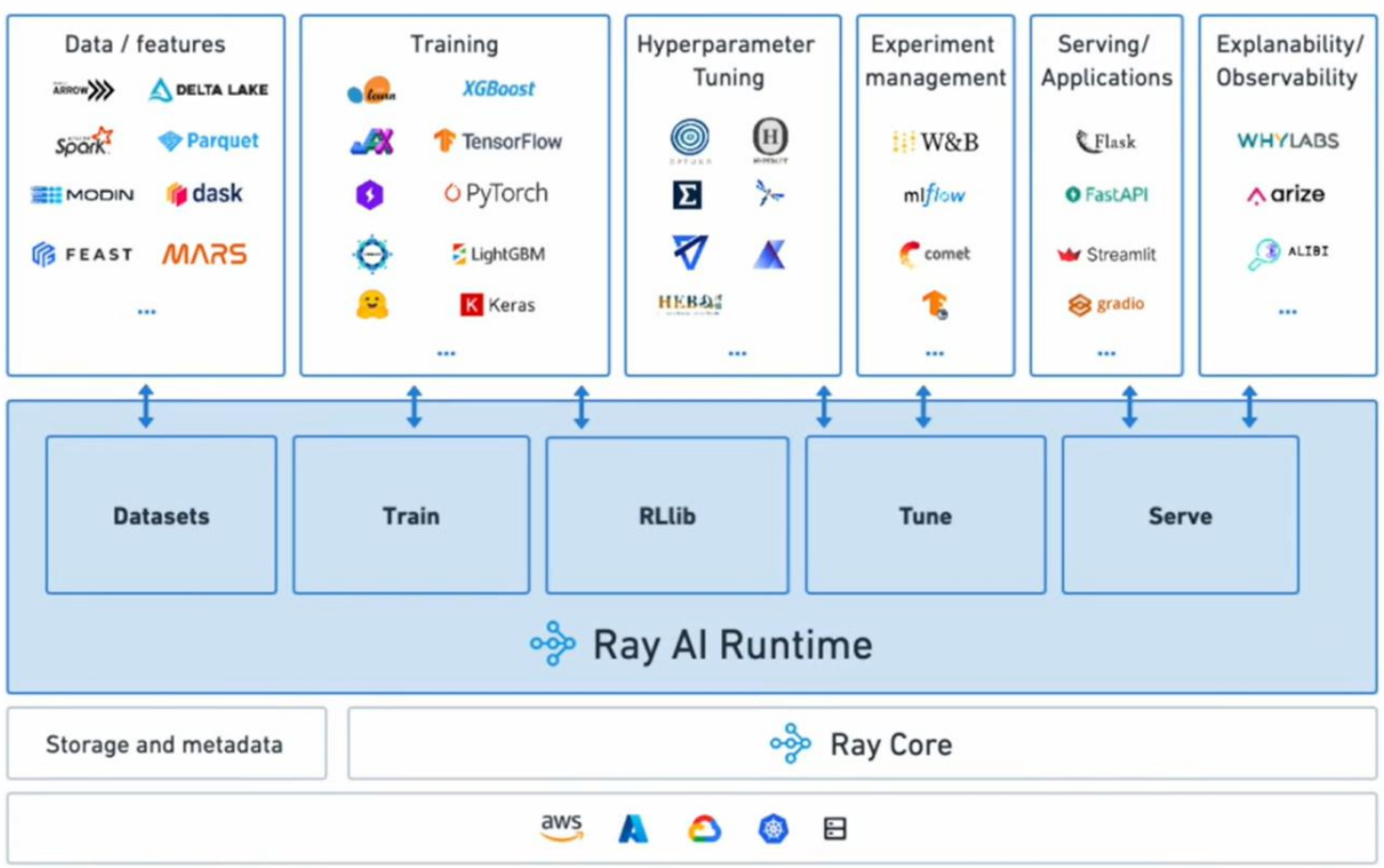


Requirements

- Fine-grained, Heterogeneous computations
- Flexible computation model
- Dynamic execution

- 대규모 클러스터링
- 새로운 프레임워크, 시뮬레이터 개발이 아닌 기존의 프레임워크, 시뮬레이터와 통합

Ray Framework



Programming Model

Name	Description
$futures = \textbf{f.remote}(args)$	Execute function f remotely. **f.remote**() can take objects or futures as inputs and returns one or more futures. This is non-blocking.
$objects = \textbf{ray.get}(futures)$	Return the values associated with one or more futures. This is blocking.
$ready\_futures = \textbf{ray.wait}(futures, k, timeout)$	Return the futures whose corresponding tasks have completed as soon as either k have completed or the timeout expires.
$actor = \textbf{Class.remote}(args)$ $futures = actor.\textbf{method.remote}(args)$	Instantiate class *Class* as a remote actor, and return a handle to it. Call a method on the remote actor and return one or more futures. Both are non-blocking.

Table 1: Ray API

Tasks (stateless)	Actors (stateful)
Fine-grained load balancing	Coarse-grained load balancing
Support for object locality	Poor locality support
High overhead for small updates	Low overhead for small updates
Efficient failure handling	Overhead from checkpointing

Table 2: Tasks vs. actors tradeoffs.

Architecture

- Driver
  - User program을 실행 시키는 프로세스
- Woker
  - Driver나 다른 Worker에 의해 Task를 실행
- Actor
  - Driver나 Woker에 의해 인스턴스화

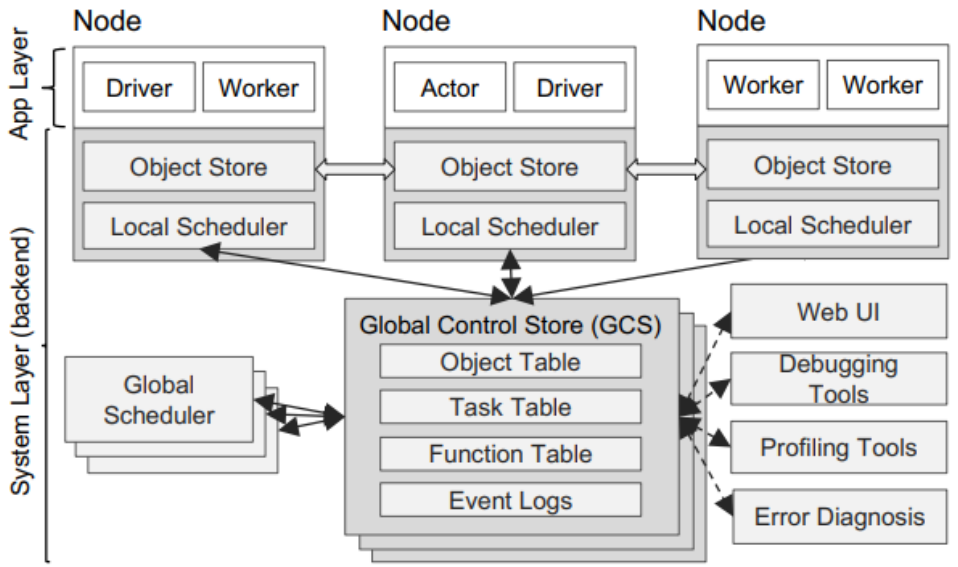


Figure 5: Ray's architecture consists of two parts: an *application* layer and a *system* layer. The application layer implements the API and the computation model described in Section 3, the system layer implements task scheduling and data management to satisfy the performance and fault-tolerance requirements.

Architecture

- Global Control Store(GCS)
  - Pub/sub 방식의 key-value store
  - Scaling, Fault-tolerance, low latency
- Bottom-Up Distributed Scheduler
  - 중앙 스케줄러 방식은 ms 작업이 초당 수백만개 발생하는 ray의 작업 환경에 적합하지 않음
  - Two-level hierarchical scheduler
    - Local scheduler, Global scheduler

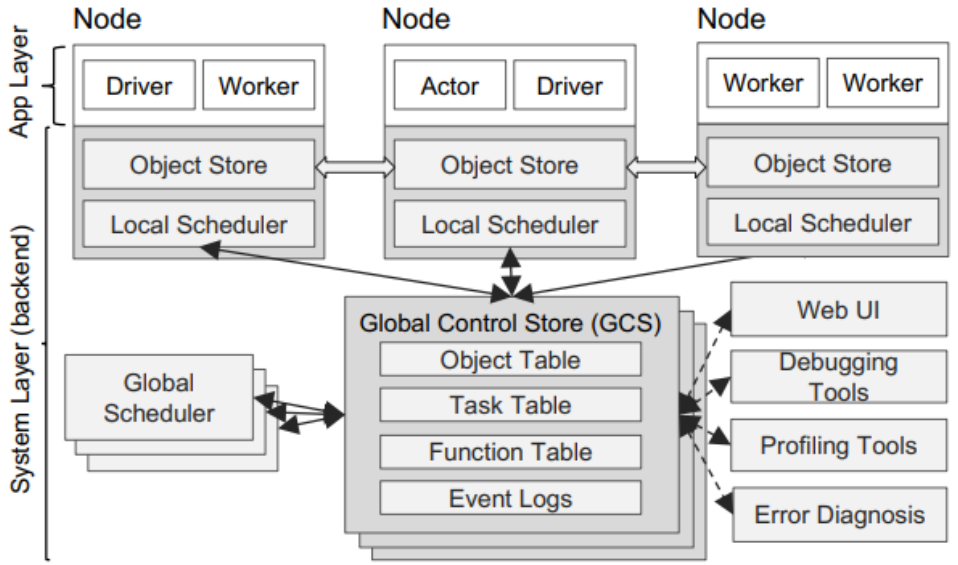


Figure 5: Ray's architecture consists of two parts: an *application* layer and a *system* layer. The application layer implements the API and the computation model described in Section 3, the system layer implements task scheduling and data management to satisfy the performance and fault-tolerance requirements.

Bottom-Up Distributed Scheduler

- Two-level hierarchical scheduler
  - Local scheduler: 노드가 여유있고 자원이 충분할 경우 스케줄링 수행
  - Global scheduler: Local scheduler가 수행할 수 없는 스케줄에 대해 스케줄링 수행
    - 노드의 작업대기 시간 + 작업의 원격 데이터 전송 시간을 고려하여 스케줄링
    - 작업대기 시간은 하트비트를 통해 지속적으로 업데이트
    - 작업 데이터의 위치와 크기는 GCS로부터 공유
    - Global scheduler에서 병목이 발생할 경우 GCS를 공유하는 복제 인스턴스 생성

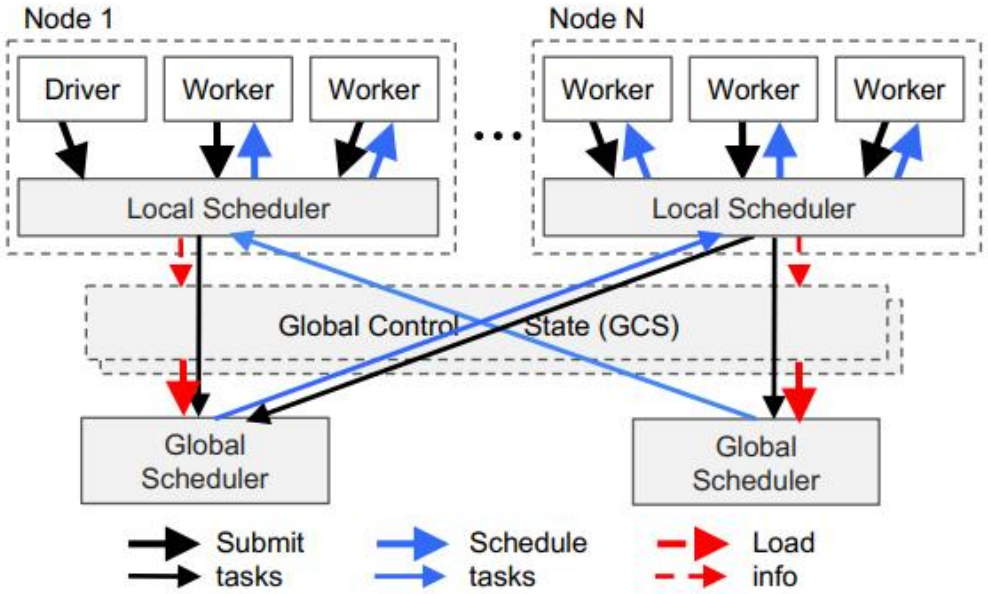


Figure 6: Bottom-up distributed scheduler. Tasks are submitted bottom-up, from drivers and workers to a local scheduler and forwarded to the global scheduler only if needed (Section 4.2.2). The thickness of each arrow is proportional to its request rate.

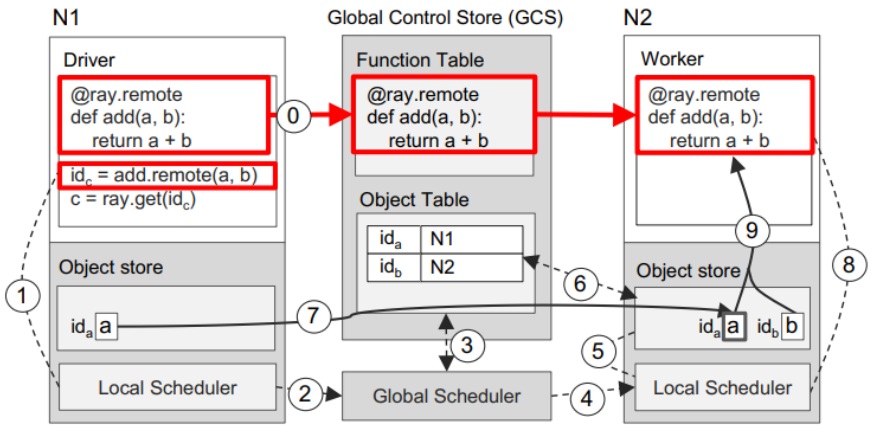
In-Memory Distributed Object Store

- 각 노드는 shared memory에 데이터를 저장하는 방식 사용
  - 동일 노드 작업들은 zero-copy data sharing 가능
- 다른 노드에 의해 시작된 작업의 경우 작업 실행 전 데이터 복제
- 아웃풋도 local에 저장

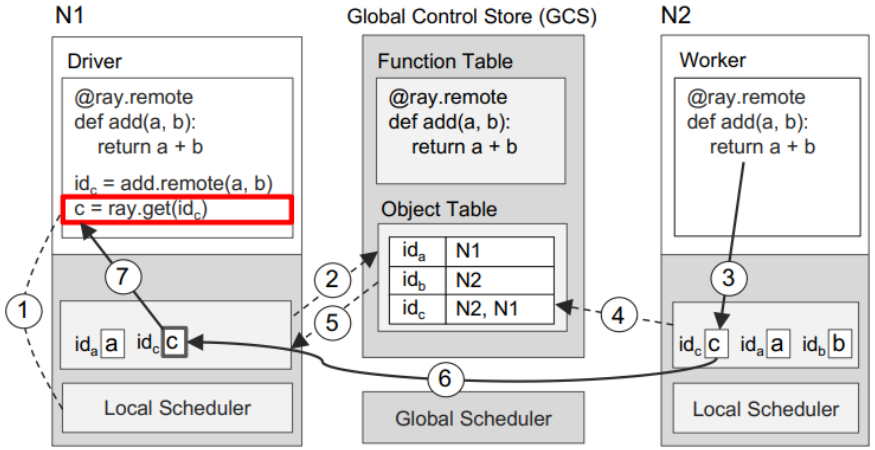
Putting Everything Together

- a+b = c 예제 (a,b,c는 scala나 matrix)
- 원격의 노드 N1과 N2 사이의 작업
- a는 N1에 위치, b는 N2에 위치

```
import ttnn
import glob
import time
import ray

@ray.remote(resources = {"TTNPU":1})
def ray_ttnpu():
    did = ttnn.get_device_ids()
    time.sleep(30)
    return did

ray.init()

print(ray.get(ray_ttnpu.remote()))
```



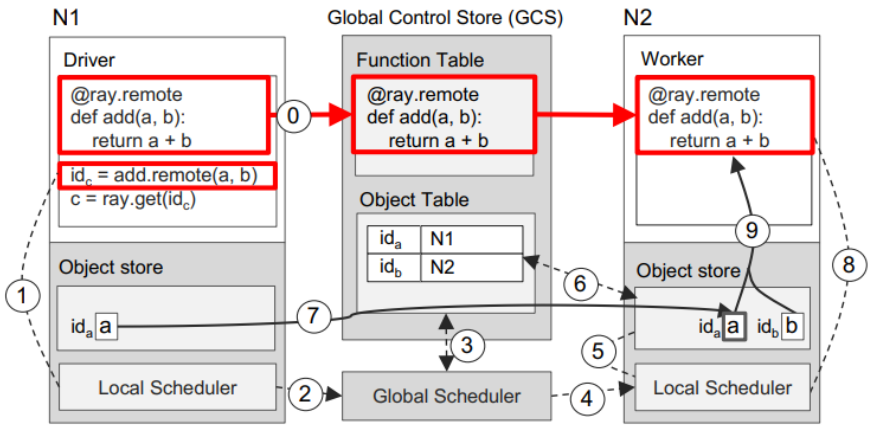
(a) Executing a task remotely



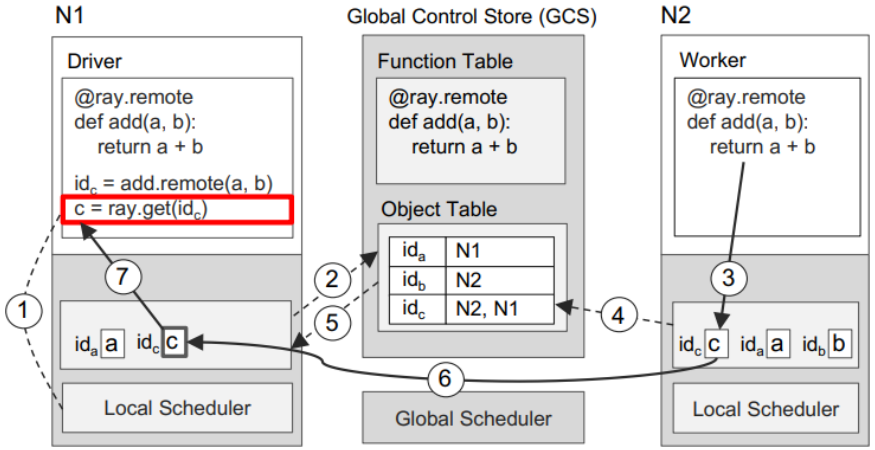
(b) Returning the result of a remote task

Putting Everything Together

- a+b = c 예제 (a,b,c는 scala나 matrix)
- 원격의 노드 N1과 N2 사이의 작업
- a는 N1에 위치, b는 N2에 위치



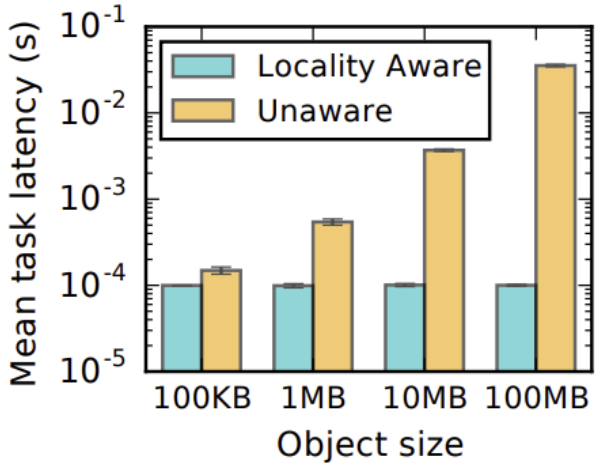
(a) Executing a task remotely



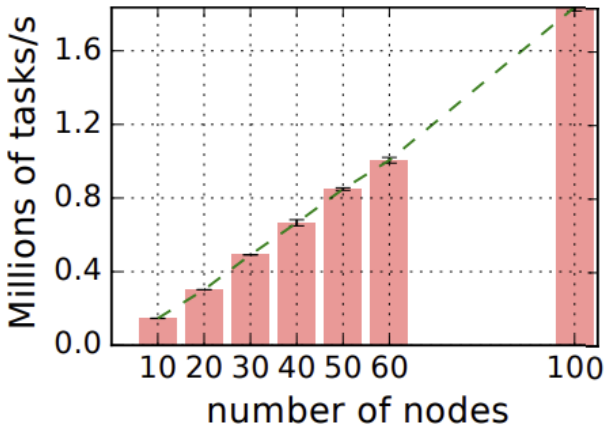
(b) Returning the result of a remote task

Evaluation

- Locality-aware task placement
- End-to-End scalability



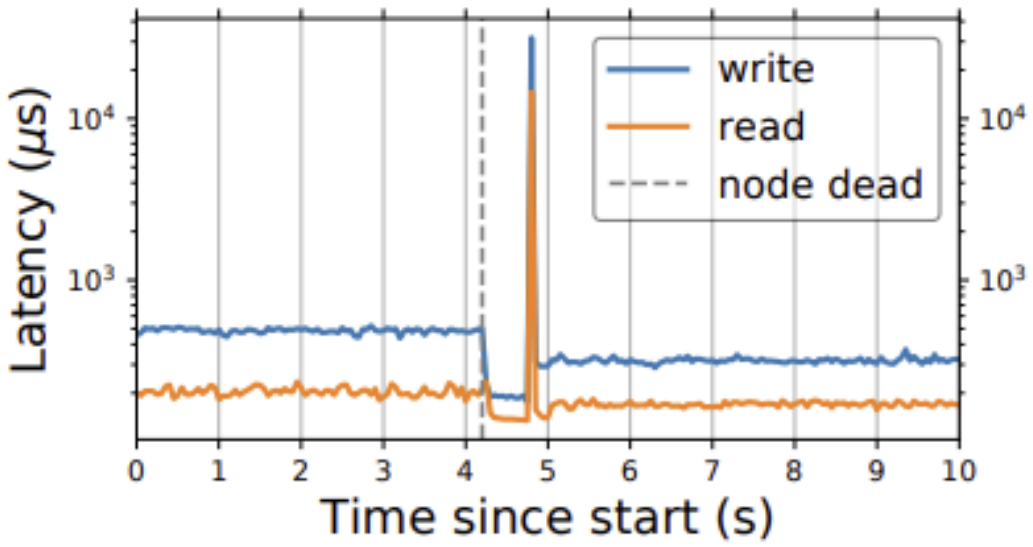
(a) Ray locality scheduling



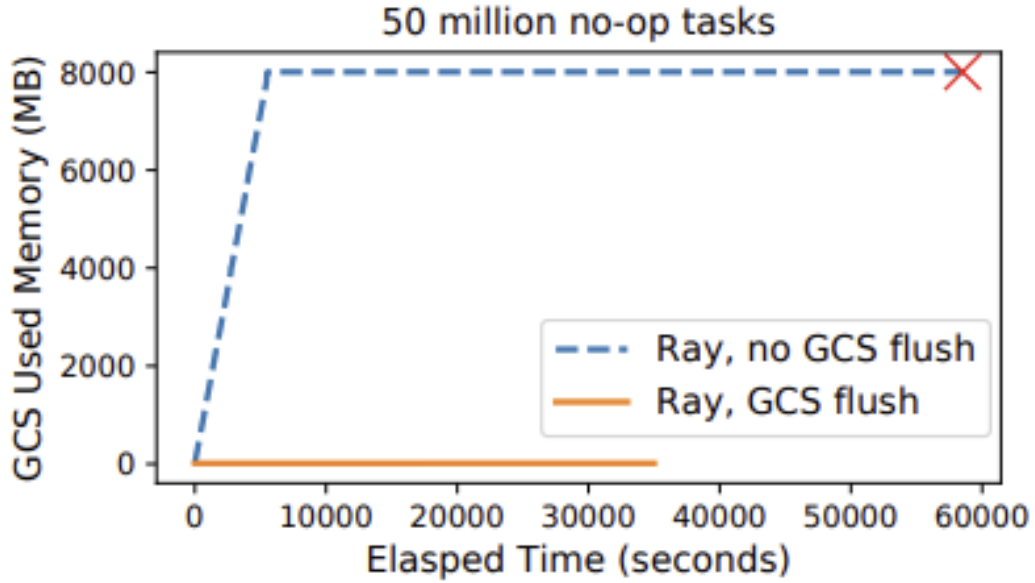
(b) Ray scalability

GCS evaluation

- Fault tolerance



- GCS Flush



TT Ray

- Accelerator Manger
  - 환경 변수를 통해 자원 할당 및 관리
- Raylet
  - CPP 기반 Ray의 Backend
  - Server 별로 생성
  - 사용 가능한 자원의 ID를 Accelerator Manger에 전달